Python WebSocket & Django Channels – Complete Notes (Beginner to Advanced)

Part 1: WebSocket Fundamentals

1. What is WebSocket?

WebSocket is a communication protocol that provides full-duplex, bidirectional communication between a client and a server over a single persistent TCP connection.

বাংলায়:

WebSocket এমন একটি communication protocol, যার মাধ্যমে Client এবং Server একই persistent connection ব্যবহার করে যেকোনো সময় একে অপরের কাছে data পাঠাতে পারে।

Key Features

- Full-duplex communication.
- Bidirectional communication.
- Persistent connection.
- Low communication overhead after handshake.
- Real-time data transfer.
- Supports text and binary messages.
- Works over TCP.

WebSocket Protocols

```
ws://example.com/ws/
wss://example.com/ws/
```

- `ws://` = Unencrypted WebSocket.
- `wss://` = Secure WebSocket using TLS.

Important: Always use WSS in production.

---

2. HTTP vs WebSocket

Feature	HTTP	WebSocket
Communication	Request-response	Bidirectional
Connection	Usually request-based	Persistent
Data transfer	Client initiates	Both sides initiate
Server push	Requires additional techniques	Native support

Real-time	Polling / SSE	Yes
Protocol	HTTP / HTTPS	WS / WSS
Overhead	Repeated request overhead	Lower after handshake

HTTP Communication

```
Client ------ Request ------> Server
Client <----- Response ------ Server
```

WebSocket Communication

```
Client <--------------------> Server
Client <--------------------> Server
Client <--------------------> Server
```

বাংলায়:
HTTP সাধারণত Client-এর request-এর পর Server response দেয়।
WebSocket connection তৈরি হওয়ার পর Client এবং Server উভয় পক্ষ স্বাধীনভাবে message পাঠাতে পারে।

---

3. How WebSocket Works

WebSocket communication সাধারণত নিচের ধাপগুলো অনুসরণ করে:

1. Client sends an HTTP request.
2. Client requests a protocol upgrade.
3. Server validates the handshake.
4. Server accepts the upgrade.
5. HTTP connection upgrades to WebSocket.
6. Persistent connection is established.
7. Client and Server exchange messages.
8. Either side can close the connection.

WebSocket Handshake

Client Request:

```
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: example-key
Sec-WebSocket-Version: 13
```

Server Response:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
```

```
  Connection: Upgrade
  Sec-WebSocket-Accept: ...
```

`101 Switching Protocols` means the server has accepted the protocol upgrade.

বাংলায়:

Handshake হলো Client এবং Server-এর মধ্যে WebSocket connection তৈরি করার প্রাথমিক প্রক্রিয়া।

---

4. WebSocket Architecture

```
              CLIENT
                 |
                 |
         WebSocket Request
                 |
                 v
              SERVER
                 |
          Accept Connection
                 |
                 v
       Persistent Connection
                 |
       ---------------------
       |                   |
       v                   v
  Send Data           Receive Data
       |                   |
       ---------------------
                 |
                 v
           Close Connection
```

Main Components

Component	Description
Client	Initiates the connection
Server	Accepts the connection
Connection	Persistent communication channel

Message	Data exchanged between client and server
Frame	Protocol-level data transfer unit
Handshake	Initial connection negotiation

---

5. Python WebSocket Libraries

Library	Purpose
websockets	Async WebSocket server and client
FastAPI	Built-in WebSocket API
Django Channels	Django real-time communication
Flask-Sock	Flask WebSocket integration
aiohttp	Async HTTP and WebSocket

Starlette	ASGI WebSocket support

Installation

Python WebSocket:

```
pip install websockets
```

FastAPI:

```
pip install fastapi uvicorn
```

Django Channels:

```
pip install django channels channels-redis
```

Daphne:

```
pip install daphne
```

Part 2: Python Async Programming

6. What is Asynchronous Programming?

Asynchronous programming allows a program to perform other tasks while waiting for an I/O operation to complete.

বাংলায়:

কোনো I/O operation শেষ হওয়ার জন্য পুরো program-কে block না রেখে অন্য কাজ চালিয়ে যাওয়ার পদ্ধতিকে Asynchronous Programming বলে।

Important Keywords

Keyword	Meaning
`async`	Defines an asynchronous function
`await`	Waits for an asynchronous operation
`asyncio`	Python asynchronous

	programming library
`create_task()`	Schedules an asynchronous task
`gather()`	Runs multiple awaitables concurrently
`sleep()`	Suspends the current coroutine temporarily

Basic Example

```
import asyncio


async def say_hello():
    print("Hello")


async def main():
    await say_hello()


asyncio.run(main())
```

asyncio.sleep()

```
import asyncio


async def main():
    print("Start")

    await asyncio.sleep(2)

    print("End")
```

```
asyncio.run(main())
```

Important: `asyncio.sleep()` does not block the entire event loop. It allows other tasks to run.

asyncio.create_task()

```
import asyncio


async def task_one():
    print("Task One Started")
    await asyncio.sleep(2)
    print("Task One Finished")


async def task_two():
    print("Task Two Started")
    await asyncio.sleep(1)
    print("Task Two Finished")


async def main():
    task1 = asyncio.create_task(task_one())
    task2 = asyncio.create_task(task_two())

    await task1
    await task2


asyncio.run(main())
```

asyncio.gather()

```
import asyncio


async def fetch_data():
    await asyncio.sleep(2)
    return "Data fetched"


async def main():
    results = await asyncio.gather(
        fetch_data(),
        fetch_data(),
        fetch_data()
    )

    print(results)


asyncio.run(main())
```

বাংলায়: Async programming বিশেষভাবে WebSocket application-এর জন্য গুরুত্বপূর্ণ, কারণ একসঙ্গে অনেকগুলো connection handle করতে হয়।

Part 3: Python WebSocket Implementation

7. Basic WebSocket Server

File: `server.py`

```
import asyncio
import websockets


async def handle_client(websocket):

    print("Client connected!")

    try:
        async for message in websocket:

            print(f"Received: {message}")

            await websocket.send(
                f"Server received: {message}"
            )

    finally:
        print("Client disconnected!")


async def main():

    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):

        print("WebSocket server running...")

        await asyncio.Future()


asyncio.run(main())
```

Explanation

Code	Meaning
`asyncio`	Handles asynchronous operations

`websockets.serve()`	Creates WebSocket server
`handle_client()`	Handles individual client
`async for`	Receives incoming messages
`websocket.send()`	Sends message
`asyncio.Future()`	Keeps server running

Note: This example uses the modern one-argument server handler API from recent versions of `websockets`.

---

8. Basic WebSocket Client

File: `client.py`

```
import asyncio
import websockets


async def main():

    uri = "ws://localhost:8765"

    async with websockets.connect(uri) as websocket:

        await websocket.send("Hello Server!")

        response = await websocket.recv()

        print("Server:", response)


asyncio.run(main())
```

Run Application

Terminal 1:

```
python server.py
```

Terminal 2:

```
python client.py
```

Output:

```
Server: Server received: Hello Server!
```

---

9. Important WebSocket Methods

Method	Description
`websockets.serve()`	Creates WebSocket server
`websockets.connect()`	Connects WebSocket client
`websocket.send()`	Sends message
`websocket.recv()`	Receives message
`websocket.close()`	Closes connection
`websocket.ping()`	Sends Ping frame
`websocket.pong()`	Sends Pong frame
`websocket.wait_closed()`	Waits for connection closure

Example

```
await websocket.send("Hello")

message = await websocket.recv()
```

```
await websocket.close()
```

---

10. WebSocket with JSON

JSON is commonly used to exchange structured data.

Server

```
import asyncio
import json
import websockets


async def handle_client(websocket):

    async for message in websocket:

        try:
            data = json.loads(message)

            username = data["username"]
            text = data["message"]

            response = {
                "username": username,
                "message": text,
                "status": "received"
            }

            await websocket.send(
                json.dumps(response)
            )

        except (json.JSONDecodeError, KeyError):

            await websocket.send(
                json.dumps({
                    "error": "Invalid message"
                })
            )


async def main():

    async with websockets.serve(
        handle_client,
        "localhost",
        8765
    ):

        await asyncio.Future()


asyncio.run(main())
```

JSON Message

```
{
    "username": "Faruk",
    "message": "Hello WebSocket"
}
```

JSON Serialization

```
json.dumps(data)
```

Converts Python object into JSON string.

JSON Deserialization

```
json.loads(message)
```

Converts JSON string into Python object.

Part 4: FastAPI WebSocket

11. WebSocket with FastAPI

FastAPI provides built-in WebSocket support through Starlette.

Installation

```
pip install fastapi uvicorn
```

Basic Implementation

File: `main.py`

```
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect


app = FastAPI()


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

    await websocket.accept()

    try:

        while True:

            message = await websocket.receive_text()

            await websocket.send_text(
                f"Message received: {message}"
            )

    except WebSocketDisconnect:

        print("Client disconnected")
```

Run Server

```
uvicorn main:app --reload
```

WebSocket URL:

```
ws://127.0.0.1:8000/ws
```

Important FastAPI WebSocket Methods

```
await websocket.accept()

await websocket.receive_text()

await websocket.receive_bytes()

await websocket.receive_json()

await websocket.send_text("Hello")

await websocket.send_bytes(b"Hello")

await websocket.send_json({
    "message": "Hello"
})

await websocket.close()
```

---

12. WebSocket Connection Lifecycle

A WebSocket connection has four main states:

1. CONNECTING
2. OPEN
3. CLOSING
4. CLOSED

State	Description
CONNECTING	Connection is being established
OPEN	Connection is active
CLOSING	Connection is closing
CLOSED	Connection is closed

Connection Lifecycle

```
await websocket.accept()

try:
```

```
    while True:
        message = await websocket.receive_text()
        await websocket.send_text(message)
except WebSocketDisconnect:
    print("Disconnected")
```

Important: In FastAPI, use `WebSocketDisconnect` to handle normal client disconnections.

---

13. Connection Manager

A Connection Manager manages multiple connected WebSocket clients.

Why Connection Manager?

- Track active clients.
- Accept connections.
- Remove disconnected clients.
- Broadcast messages.
- Manage client connections.

Implementation

```
from fastapi import FastAPI, WebSocket
from starlette.websockets import WebSocketDisconnect

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active_connections = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        if websocket in self.active_connections:
            self.active_connections.remove(websocket)

    async def send_personal_message(
```

```
        self,
        message: str,
        websocket: WebSocket
    ):

        await websocket.send_text(message)


    async def broadcast(self, message: str):

        for connection in self.active_connections.copy():

            try:

                await connection.send_text(message)

            except Exception:

                self.disconnect(connection)


manager = ConnectionManager()


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):

    await manager.connect(websocket)

    try:

        while True:

            message = await websocket.receive_text()

            await manager.broadcast(message)

    except WebSocketDisconnect:

        manager.disconnect(websocket)
```

Important: This in-memory manager is suitable for a basic single-process demonstration. Multiple application workers require shared coordination, such as Redis.

---

14. Broadcasting Messages

Broadcasting means sending one message to all connected clients.

```
async def broadcast(self, message: str):

    for connection in self.active_connections:

        await connection.send_text(message)
```

Use Cases

- Group chat.
- Live notifications.
- Multiplayer games.
- Real-time dashboards.
- Live sports updates.

Personal Message vs Broadcast

Personal Message	Broadcast
One client	Multiple clients
Private communication	Group communication
Individual delivery	Group delivery

---

15. WebSocket Authentication

Authentication ensures that only authorized users can connect.

Common Authentication Methods

- JWT Authentication.
- Session Authentication.
- OAuth2.
- API Token.

JWT Authentication Concept

```
User Login
    |
    v
Generate JWT
    |
    v
Client Receives JWT
    |
    v
WebSocket Connection
    |
    v
Validate JWT
    |
    v
Accept or Reject
```

FastAPI Demonstration

```
from fastapi import FastAPI, WebSocket, Query


app = FastAPI()
```

```
@app.websocket("/ws")
async def websocket_endpoint(
    websocket: WebSocket,
    token: str = Query(...)
):

    if token != "secret-token":

        await websocket.close(code=1008)

        return

    await websocket.accept()

    while True:

        message = await websocket.receive_text()

        await websocket.send_text(message)
```

Important: This is only a demonstration. Real applications must validate JWT signatures, expiration, issuer, audience and permissions.
Avoid exposing access tokens in query strings because URLs may appear in logs.

---

16. WebSocket Authorization

Authentication and authorization are different.
Authentication: Who are you?
Authorization: What are you allowed to do?
Example:

- User A can access Room 1.
- User B can access Room 2.
- Admin can access both rooms.

Always validate permissions before allowing access to a room or performing an action.

---

17. WebSocket Ping and Pong

Ping and Pong are WebSocket control frames.

Ping

A control frame used to check connection availability.

Pong

A response to a Ping frame.

Why Heartbeat?

- Detect dead connections.
- Maintain connection health.
- Detect network failures.
- Improve reliability.

Python WebSocket Example

```
async with websockets.serve(
    handle_client,
    "localhost",
    8765,
    ping_interval=20,
    ping_timeout=20
):

    await asyncio.Future()
```

- `ping_interval` : Time between automatic pings.
- `ping_timeout` : Maximum waiting time for pong.

Important: Browser JavaScript WebSocket API does not expose direct Ping/Pong control-frame methods.

---

18. WebSocket Reconnection

Sometimes connections fail because of:

- Internet disconnection.
- Server restart.
- Network timeout.
- Proxy interruption.

Clients should reconnect when appropriate.

Python Reconnection Example

```
import asyncio
import websockets


async def connect():

    uri = "ws://localhost:8765"

    while True:

        try:

            async with websockets.connect(uri) as websocket:

                print("Connected!")

                while True:

                    message = await websocket.recv()

                    print(message)

        except (
            websockets.exceptions.ConnectionClosed,
            OSError
        ) as error:
```

```
            print("Connection failed:", error)

            await asyncio.sleep(5)


asyncio.run(connect())
```

Production applications should use:

- Exponential backoff.
- Maximum retry limits.
- Random jitter.
- Authentication refresh.
- Proper connection cleanup.

Part 5: Redis and WebSocket

19. Redis Pub/Sub with WebSocket

Redis Pub/Sub allows multiple application instances to exchange messages.

Why Redis?

Consider an application running multiple server instances:

```
             CLIENTS
                |
           LOAD BALANCER
             /     \
            /       \
      SERVER 1   SERVER 2
            \       /
             \     /
              REDIS
```

Without shared messaging, a message received by Server 1 may not reach clients connected to Server 2.

Redis Pub/Sub helps distribute messages between application instances.

Installation

```
pip install redis
```

Redis Connection

```
import redis.asyncio as redis


client = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)
```

Publish

```
await client.publish(
    "chat",
```

```
    "Hello Everyone"
)
```

Subscribe

```
pubsub = client.pubsub()

await pubsub.subscribe("chat")

async for message in pubsub.listen():

    if message["type"] == "message":

        print(message["data"])
```

Important: Redis Pub/Sub is not durable. Disconnected subscribers can miss messages. For persistent event delivery, consider Redis Streams or another message broker.

---

20. WebSocket Rooms

A room allows a group of users to communicate with each other.

Example

```
Room 1:
    User A
    User B
    User C

Room 2:
    User D
    User E
```

Room Manager Concept

```
class RoomManager:

    def __init__(self):

        self.rooms = {}


    async def join_room(self, room_id, websocket):

        await websocket.accept()

        self.rooms.setdefault(room_id, []).append(websocket)


    def leave_room(self, room_id, websocket):

        if room_id in self.rooms:

            if websocket in self.rooms[room_id]:

                self.rooms[room_id].remove(websocket)
```

```
            if not self.rooms[room_id]:
                del self.rooms[room_id]

    async def send_to_room(self, room_id, message):
        for connection in self.rooms.get(room_id, []).copy():
            await connection.send_text(message)
```

Important: Production room managers need disconnect cleanup, authorization, message validation and concurrent-send handling.

---

21. WebSocket Message Types

WebSocket supports text and binary messages.

Text Message

```
await websocket.send_text("Hello")
```

Binary Message

```
await websocket.send_bytes(b"Hello")
```

JSON Message

```
await websocket.send_json({
    "username": "Faruk",
    "message": "Hello"
})
```

Receive Messages

```
text = await websocket.receive_text()

binary = await websocket.receive_bytes()

data = await websocket.receive_json()
```

---

22. WebSocket Close Codes

Code	Meaning
1000	Normal closure
1001	Going away
1002	Protocol error
1003	Unsupported data

1006	Abnormal closure (reserved)
1007	Invalid frame payload data
1008	Policy violation
1009	Message too big
1011	Internal server error

Example

```
await websocket.close(code=1000)
```

Part 6: WebSocket Security and Performance

23. WebSocket Security Best Practices

1. Always use WSS.
2. Validate authentication tokens.
3. Validate Origin headers.
4. Validate user permissions.
5. Implement rate limiting.
6. Set message size limits.
7. Validate incoming JSON.
8. Handle exceptions properly.
9. Prevent unauthorized room access.
10. Avoid logging sensitive data.
11. Implement connection limits.
12. Protect against denial-of-service attacks.

Message Validation Example

```
from pydantic import BaseModel, ValidationError


class ChatMessage(BaseModel):

    username: str

    message: str


def validate_message(data):

    try:

        return ChatMessage.model_validate(data)
```

```
    except ValidationError:

        return None
```

---

24. WebSocket vs Server-Sent Events (SSE)

Feature	WebSocket	SSE
Communication	Bidirectional	Server to client
Protocol	WebSocket	HTTP
Real-time	Yes	Yes
Client messages	Yes	Separate HTTP request
Reconnection	Application-managed	Built-in browser support
Use case	Chat, gaming	Live feeds, notifications

---

25. Polling vs Long Polling vs SSE vs WebSocket

Polling

Client নির্দিষ্ট সময় পরপর Server-কে request পাঠায়।

```
Client ---> Server
Client ---> Server
Client ---> Server
```

Example:

```
setInterval(async () => {

    const response = await fetch("/api/updates/");

    const data = await response.json();

    console.log(data);
```

```
}, 5000);
```

Long Polling

Client request পাঠায়। Server data পাওয়া পর্যন্ত বা timeout হওয়া পর্যন্ত request ধরে রাখে।
Response পাওয়ার পর Client আবার request পাঠায়।

SSE

Server থেকে Client-এর দিকে একমুখী real-time event stream পাঠানো হয়।

WebSocket

Client এবং Server উভয় দিক থেকেই message পাঠানো যায়।

Comparison

Feature	Polling	Long Polling	SSE	WebSocket
Direction	Client request	Server response	Server to Client	Both directions
Persistent	No	Request-based	Yes	Yes
Real-time	Limited	Better	Yes	Yes
Browser support	Yes	Yes	Yes	Yes
Best for	Simple updates	Older real-time systems	Notifications	Chat and gaming

---

26. WebSocket Performance Optimization

1. Use asynchronous programming.
2. Avoid blocking operations.
3. Use Redis for distributed messaging.
4. Limit message sizes.
5. Use efficient JSON serialization.
6. Implement connection limits.
7. Use appropriate heartbeat settings.
8. Avoid unnecessary broadcasting.
9. Monitor active connections.
10. Use load balancing.
11. Handle backpressure.
12. Optimize database queries.
13. Use pagination for chat history.
14. Release disconnected client resources.

---

27. WebSocket Scaling

ধরো, তোমার application-এ ১০,০০০ concurrent users রয়েছে।

একটি server দিয়ে সব connection পরিচালনা করা কঠিন হতে পারে।

তখন multiple servers ব্যবহার করা যায়।

Architecture

```
                USERS
                  |
            LOAD BALANCER
                  |
      ---------------------
      |         |         |
Server 1  Server 2  Server 3
      |         |         |
      ---------------------
                |
              REDIS
                |
            DATABASE
```

Important Components

Load Balancer

Incoming connection বিভিন্ন server-এ distribute করে।

Multiple Application Servers

একাধিক server connection handle করে।

Redis

Server-এর মধ্যে event বা message exchange করতে সাহায্য করে।

Database

Permanent information সংরক্ষণ করে।

Sticky Sessions

কিছু deployment setup-এ একই client-এর connection নির্দিষ্ট server-এর সঙ্গে ধরে রাখার প্রয়োজন হতে পারে।

তবে WebSocket handshake সম্পন্ন হওয়ার পর connection সাধারণত সেই server-এর সঙ্গেই থাকে।

---

28. WebSocket Deployment

Production deployment-এর জন্য প্রয়োজনীয় বিষয়:

- ASGI server.
- Nginx reverse proxy.
- HTTPS/TLS.
- Redis.
- Load balancer.
- Authentication.
- Monitoring.
- Logging.
- Connection limits.

Uvicorn Command

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

Nginx Configuration

```
location /ws/ {

    proxy_pass http://127.0.0.1:8000;

    proxy_http_version 1.1;

    proxy_set_header Upgrade $http_upgrade;

    proxy_set_header Connection "upgrade";

    proxy_set_header Host $host;

    proxy_read_timeout 3600s;
}
```

Part 7: Django Channels Fundamentals

29. What is Django Channels?

Django Channels হলো Django-এর একটি extension, যা WebSocket এবং অন্যান্য asynchronous communication protocol support করে।
সাধারণ Django HTTP request-response পদ্ধতিতে কাজ করে।
Django Channels ব্যবহার করে Client এবং Server-এর মধ্যে persistent connection তৈরি করা যায়।

Django Channels Use Cases

- Real-time chat application.
- Live notifications.
- Live dashboards.
- Typing indicators.
- Online user presence.
- Live tracking.
- Multiplayer games.
- Real-time collaboration.

---

30. Important Django Channels Concepts

1. ASGI

ASGI stands for Asynchronous Server Gateway Interface.
এটি asynchronous Python web application চালানোর standard interface।
Django Channels সাধারণত `asgi.py` ব্যবহার করে।

2. WSGI vs ASGI

WSGI	ASGI
Traditional synchronous interface	Asynchronous-capable interface

Primarily HTTP	HTTP and WebSocket
Request-response oriented	Supports long-lived connections
Blocking execution model	Supports async execution

3. Consumer

Consumer হলো Django Channels-এর এমন একটি component, যা WebSocket connection এবং message handle করে।

4. Routing

Routing নির্ধারণ করে কোন WebSocket URL কোন Consumer-এর কাছে যাবে।

5. Channel Layer

Channel Layer বিভিন্ন Consumer-এর মধ্যে message পাঠানোর ব্যবস্থা করে।

6. Redis

Redis একটি in-memory data store। Django Channels-এর production channel layer হিসেবে Redis বহুল ব্যবহৃত হয়।

7. Group

Group-এর সাহায্যে একাধিক connected client-কে একই message পাঠানো যায়।

Part 8: Django Channels Project Setup

31. Installation

প্রথমে Virtual Environment তৈরি করো।

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Packages install করো:

```
pip install django channels channels-redis daphne
```

Create Django Project

```
django-admin startproject config .
```

Create App:

```
python manage.py startapp chat
```

Project Structure

```
project/
│
├── manage.py
│
├── config/
│   ├── __init__.py
```

```
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── routing.py
│
└── chat/
    ├── __init__.py
    ├── consumers.py
    ├── routing.py
    ├── views.py
    ├── urls.py
    ├── models.py
    └── tests.py
```

---

32. Configure settings.py

File: `config/settings.py`

```
INSTALLED_APPS = [

    "daphne",

    "django.contrib.admin",

    "django.contrib.auth",

    "django.contrib.contenttypes",

    "django.contrib.sessions",

    "django.contrib.messages",

    "django.contrib.staticfiles",

    "channels",

    "chat",
]


ASGI_APPLICATION = "config.asgi.application"
```

Redis Channel Layer Configuration

```
CHANNEL_LAYERS = {

    "default": {

        "BACKEND": "channels_redis.core.RedisChannelLayer",

        "CONFIG": {

            "hosts": [
                "redis://127.0.0.1:6379/0"
            ],
```

```
        },
    },
}
```

Important:
এই configuration-এর জন্য Redis server চালু থাকতে হবে।
Redis configuration ছাড়া basic development-এর জন্য InMemoryChannelLayer ব্যবহার করা যায়।

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels.layers.InMemoryChannelLayer",
    },
}
```

InMemoryChannelLayer শুধু local development এবং testing-এর জন্য উপযুক্ত। Multi-process production deployment-এর জন্য এটি ব্যবহার করা উচিত নয়।

---

33. Create Your First Consumer

File: `chat/consumers.py`
Consumer-এর গুরুত্বপূর্ণ methods:

Method	Description
`connect()`	Client connect করলে execute হয়
`disconnect()`	Client disconnect হলে execute হয়
`receive()`	Client message পাঠালে execute হয়
`send()`	Client-কে message পাঠায়
`accept()`	WebSocket connection গ্রহণ করে
`close()`	Connection বন্ধ করে

Echo Consumer

```
from channels.generic.websocket import AsyncWebsocketConsumer


class EchoConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()
```

```
        await self.send(
            text_data="WebSocket Connected!"
        )


    async def disconnect(self, close_code):

        print("WebSocket disconnected:", close_code)


    async def receive(
        self,
        text_data=None,
        bytes_data=None
    ):

        if text_data is not None:

            await self.send(
                text_data=f"You sent: {text_data}"
            )
```

---

34. WebSocket Routing

File: `chat/routing.py`

```
from django.urls import path

from .consumers import EchoConsumer


websocket_urlpatterns = [

    path(
        "ws/echo/",
        EchoConsumer.as_asgi()
    ),

]
```

Why as_asgi()?

`as_asgi()` Consumer class-কে ASGI application callable-এ convert করে।
এটি WebSocket routing-এ Consumer ব্যবহার করার জন্য প্রয়োজন।

---

35. Configure ASGI Application

File: `config/asgi.py`

```
import os

from django.core.asgi import get_asgi_application

from channels.routing import (
    ProtocolTypeRouter,
```

```
    URLRouter
)

from channels.auth import AuthMiddlewareStack


os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)


django_asgi_app = get_asgi_application()


from chat.routing import websocket_urlpatterns


application = ProtocolTypeRouter({

    "http": django_asgi_app,

    "websocket": AuthMiddlewareStack(

        URLRouter(
            websocket_urlpatterns
        )

    ),

})
```

Explanation

Component	Purpose
`get_asgi_application()`	Initializes Django ASGI application
`ProtocolTypeRouter`	Routes HTTP and WebSocket protocols
`URLRouter`	Matches WebSocket URLs
`AuthMiddlewareStack`	Provides session-based user authentication
`websocket_urlpatterns`	Defines WebSocket routes

Important: `get_asgi_application()` initialize করার পর Consumer এবং model-related imports করা ভালো, যাতে Django application setup সম্পন্ন হয়।

---

36. Create a Test HTML Page

File:

chat/templates/chat/index.html

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Django Channels Echo</title>

</head>

<body>

    <h1>WebSocket Echo Test</h1>

    <input id="messageInput" type="text">

    <button onclick="sendMessage()">
        Send
    </button>

    <ul id="messages"></ul>

    <script>

        const scheme =
            location.protocol === "https:" ? "wss" : "ws";

        const socket = new WebSocket(
            `${scheme}://${location.host}/ws/echo/`
        );


        socket.onopen = function () {

            console.log("Connected");

        };


        socket.onmessage = function (event) {

            const li = document.createElement("li");

            li.textContent = event.data;

            document
                .getElementById("messages")
                .appendChild(li);
```

```
        };


        socket.onclose = function () {

            console.log("Disconnected");

        };


        socket.onerror = function (error) {

            console.error("WebSocket error:", error);

        };


        function sendMessage() {

            const input =
                document.getElementById("messageInput");

            if (socket.readyState === WebSocket.OPEN) {

                socket.send(input.value);

                input.value = "";

            }

        }

    </script>

</body>

</html>
```

Create View

File: `chat/views.py`

```
from django.shortcuts import render


def home(request):

    return render(
        request,
        "chat/index.html"
    )
```

App URLs

File: `chat/urls.py`

```
from django.urls import path

from . import views


urlpatterns = [

    path("", views.home, name="home"),

]
```

Project URLs

File: `config/urls.py`

```
from django.contrib import admin

from django.urls import path, include


urlpatterns = [

    path("admin/", admin.site.urls),

    path("", include("chat.urls")),

]
```

Run Project

```
python manage.py migrate
```

```
python manage.py runserver
```

Browser:

```
http://127.0.0.1:8000/
```

Part 9: Sync vs Async Consumers

37. Synchronous Consumer

```
from channels.generic.websocket import WebsocketConsumer


class MyConsumer(WebsocketConsumer):

    def connect(self):

        self.accept()


    def receive(self, text_data=None, bytes_data=None):

        self.send(
            text_data="Hello from sync consumer!"
```

```
        )


    def disconnect(self, close_code):

        pass
```

38. Asynchronous Consumer

```
from channels.generic.websocket import AsyncWebsocketConsumer


class MyAsyncConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        await self.accept()


    async def receive(self, text_data=None, bytes_data=None):

        await self.send(
            text_data="Hello from async consumer!"
        )


    async def disconnect(self, close_code):

        pass
```

Difference

Sync Consumer	Async Consumer
Synchronous methods	Asynchronous methods
Blocking operations may occur	Supports non-blocking I/O
Normal synchronous code	Async operations
No await needed	Uses async and await

Important:
Async Consumer-এর ভিতরে synchronous database operations সরাসরি চালালে event loop block হতে পারে।
Async ORM methods অথবা `database_sync_to_async` ব্যবহার করো।

Part 10: Django Channels JSON Communication

39. JSON Messages

Client এবং Server-এর মধ্যে structured data পাঠানোর জন্য JSON ব্যবহার করা হয়।

JSON Consumer

```
import json

from channels.generic.websocket import AsyncWebsocketConsumer


class JsonConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        await self.accept()


    async def receive(self, text_data=None, bytes_data=None):

        try:

            data = json.loads(text_data or "{}")

        except json.JSONDecodeError:

            await self.send(
                text_data=json.dumps({
                    "error": "Invalid JSON"
                })
            )

            return


        message = data.get("message", "")


        await self.send(
            text_data=json.dumps({
                "message": message,
                "status": "success"
            })
        )
```

Client JavaScript

```
socket.send(JSON.stringify({

    message: "Hello Django"

}));
```

Part 11: Channel Layer and Redis

40. What is Channel Layer?

Channel Layer হলো Django Channels-এর messaging system।

এটি বিভিন্ন Consumer-এর মধ্যে message এবং event আদান-প্রদান করতে সাহায্য করে।

Why Channel Layer?

ধরো, একটি chat room-এ অনেক user connected আছে।

একজন user message পাঠালে অন্য user-দের কাছে message পাঠাতে হবে।

Channel Layer এই কাজটি করতে সাহায্য করে।

In-Memory Channel Layer

Development এবং testing-এর জন্য ব্যবহার করা যায়।

Redis Channel Layer

Multiple application process বা server-এর মধ্যে message exchange করার জন্য Redis ব্যবহার করা হয়।

Installation

```
pip install channels-redis
```

Redis Configuration

```
CHANNEL_LAYERS = {

    "default": {

        "BACKEND": "channels_redis.core.RedisChannelLayer",

        "CONFIG": {

            "hosts": [
                "redis://127.0.0.1:6379/0"
            ],

        },

    },

}
```

Redis Responsibilities

- Cross-server communication.
- Group broadcasting.
- Consumer-to-consumer messaging.
- Temporary message distribution.

Important: Redis Channel Layer permanent message storage-এর বিকল্প নয়।

Chat history, user information এবং order information স্থায়ীভাবে রাখতে PostgreSQL বা অন্য database ব্যবহার করতে হবে।

Part 12: Django Channels Group Chat

41. What is Broadcasting?

Broadcasting হলো একটি message একাধিক connected client-এর কাছে পাঠানো।

Example:

একটি chat room-এ ১০ জন user connected আছে।

একজন user message পাঠালে অন্য user-রা সেই message পাবে।

Group Methods

Method	Purpose
`group_add()`	Consumer-কে group-এ যুক্ত করে
`group_discard()`	Consumer-কে group থেকে সরিয়ে দেয়
`group_send()`	Group-এর সব member-এর কাছে event পাঠায়
`channel_name`	Individual Consumer-এর unique channel name

Complete Group Chat Consumer

File: `chat/consumers.py`

```
import json

from channels.generic.websocket import AsyncWebsocketConsumer


class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        self.room_name = self.scope["url_route"]["kwargs"]["room_name"]

        self.room_group_name = f"chat_{self.room_name}"


        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )


        await self.accept()


    async def disconnect(self, close_code):

        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )


    async def receive(
        self,
        text_data=None,
        bytes_data=None
    ):
```

```
        try:
            data = json.loads(text_data or "{}")
        except json.JSONDecodeError:
            return

        message = data.get("message", "")

        if not isinstance(message, str) or not message.strip():
            return

        await self.channel_layer.group_send(
            self.room_group_name,
            {
                "type": "chat_message",
                "message": message.strip()
            }
        )

    async def chat_message(self, event):
        await self.send(
            text_data=json.dumps({
                "message": event["message"]
            })
        )
```

Room Routing

File: `chat/routing.py`

```
from django.urls import path
from .consumers import ChatConsumer

websocket_urlpatterns = [
```

```
    path(
        "ws/chat/<str:room_name>/",
        ChatConsumer.as_asgi()
    ),

]
```

Important Event Type

```
{

    "type": "chat_message",

    "message": "Hello Everyone"

}
```

এখানে `type` হলো handler method-এর নাম নির্ধারণ করে।
`type="chat_message"` হলে `chat_message()` method execute হবে।
Important: এই example-টি basic demonstration। Production application-এ authentication, room permission, rate limiting এবং message persistence যোগ করতে হবে।

---

42. WebSocket Client for Chat

HTML JavaScript:

```
<input id="messageInput" type="text">

<button onclick="sendMessage()">
    Send
</button>

<ul id="messages"></ul>


<script>

    const scheme =
        location.protocol === "https:" ? "wss" : "ws";


    const roomName = "general";


    const socket = new WebSocket(

        `${scheme}://${location.host}/ws/chat/${roomName}/`

    );


    socket.onopen = function () {

        console.log("Connected");
```

```
    };


    socket.onmessage = function (event) {

        const data = JSON.parse(event.data);

        const li = document.createElement("li");

        li.textContent = data.message;

        document
            .getElementById("messages")
            .appendChild(li);

    };


    socket.onclose = function () {

        console.log("Disconnected");

    };


    function sendMessage() {

        const input =
            document.getElementById("messageInput");


        if (socket.readyState === WebSocket.OPEN) {

            socket.send(JSON.stringify({

                message: input.value

            }));

            input.value = "";

        }

    }

</script>
```

একই room name দিয়ে একাধিক browser tab খুলে test করো।
একটি tab থেকে message পাঠালে অন্য tab-গুলোও message পাবে।

Part 13: Authentication and Authorization

43. Django Channels Scope

`self.scope`-এ connection সম্পর্কিত তথ্য থাকে।
উদাহরণ:

```
user = self.scope["user"]

room_name = self.scope["url_route"]["kwargs"]["room_name"]
```

Important Scope Information

Scope Property	Meaning
`scope["type"]`	Connection protocol type
`scope["user"]`	Authenticated user
`scope["path"]`	Request path
`scope["headers"]`	Connection headers
`scope["url_route"]`	URL routing information
`scope["query_string"]`	Query string bytes

Session Authentication

`AuthMiddlewareStack` ব্যবহার করলে WebSocket scope-এ session-based authenticated user পাওয়া যায়।

Authentication Example

```
from channels.generic.websocket import AsyncWebsocketConsumer


class PrivateChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):

        user = self.scope["user"]


        if not user.is_authenticated:

            await self.close()

            return


        await self.accept()
```

Important:
Authentication এবং authorization এক জিনিস নয়।
Protected chat room-এর ক্ষেত্রে user authenticated হওয়ার পাশাপাশি room access permission আছে কি না সেটিও check করতে হবে।

---

44. JWT Authentication

JWT ব্যবহার করলে token validate করতে হবে।

Authentication Flow

```
Client
   |
   | JWT Token
   |
   v
WebSocket Server
   |
   | Validate Token
   |
   v
Authenticated User
```

Important Browser Limitation:

Browser-এর native WebSocket constructor arbitrary custom headers সেট করার সুযোগ দেয় না।

তাই browser client-এর ক্ষেত্রে secure session cookie অথবা carefully designed token authentication flow ব্যবহার করা যায়।

URL query string-এ token পাঠালে server, proxy বা access log-এ token থেকে যেতে পারে।

তাই এটি এড়িয়ে চলা বা অত্যন্ত সতর্কভাবে পরিচালনা করা উচিত।

Part 14: Database Operations in Async Consumers

45. Django Models

Async Consumer-এ synchronous ORM call সরাসরি ব্যবহার করা এড়িয়ে চলতে হবে।

File: `chat/models.py`

```
from django.conf import settings

from django.db import models


class Room(models.Model):

    name = models.CharField(
        max_length=100,
        unique=True
    )


    def __str__(self):

        return self.name



class Message(models.Model):

    room = models.ForeignKey(
        Room,
```

```
        on_delete=models.CASCADE,
        related_name="messages"
    )

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE
    )

    content = models.TextField()

    created_at = models.DateTimeField(
        auto_now_add=True
    )


    def __str__(self):

        return f"{self.user} - {self.room}"
```

Migration

```
python manage.py makemigrations
```

```
python manage.py migrate
```

---

46. Database Operations with database_sync_to_async

`database_sync_to_async` synchronous database operations-কে async-compatibleভাবে চালাতে সাহায্য করে।

Database Helper

```
from channels.db import database_sync_to_async

from .models import Message, Room


@database_sync_to_async
def save_message(room_name, user, content):

    room = Room.objects.get(name=room_name)

    return Message.objects.create(

        room=room,

        user=user,

        content=content

    )


@database_sync_to_async
def get_recent_messages(room_name):
```

```
    room = Room.objects.get(name=room_name)
    return list(
        Message.objects.filter(room=room)
        .select_related("user")
        .order_by("-created_at")[:50]
    )
```

Consumer থেকে ব্যবহার

```
message = await save_message(
    self.room_name,
    self.scope["user"],
    "Hello!"
)
```

Important:
Database access করার আগে user authenticated কি না এবং room-এ তার permission আছে কি না যাচাই করতে হবে। `Room.DoesNotExist`-এর মতো database errors-ও handle করতে হবে।

Part 15: Real-Time Features

47. Real-Time Notifications

Notification event পাঠানোর ধারণা:

```
await self.channel_layer.group_send(
    f"user_{user_id}",
    {
        "type": "notification_message",
        "message": "You have a new notification"
    }
)
```

Consumer Event Handler

```
async def notification_message(self, event):
    await self.send(
```

```
        text_data=json.dumps({
            "type": "notification",
            "message": event["message"]
        })
    )
```

Notification group-এ user-কে যুক্ত করার আগে authentication ও ownership যাচাই করতে হবে।

---

48. Typing Indicator

Typing indicator ব্যবহার করে বোঝা যায় কোনো user message লিখছে কি না।

Client Event

```
socket.send(JSON.stringify({
    type: "typing",
    is_typing: true
}));
```

Server Event Handling

```
if data.get("type") == "typing":
    await self.channel_layer.group_send(
        self.room_group_name,
        {
            "type": "typing_event",
            "is_typing": bool(data.get("is_typing"))
        }
    )
    return
```

Event Handler

```
async def typing_event(self, event):
    await self.send(
        text_data=json.dumps({
            "type": "typing",
```

```
            "is_typing": event["is_typing"]

        })

    )
```

Production application-এ typing event-এর সঙ্গে sender identity যোগ করো এবং event spam সীমিত করো।

---

49. Online User Presence

Online presence বলতে বোঝায় কোন user বর্তমানে application-এ active আছে।

Basic Process

1. User connect করলে presence record update করা হয়।
2. Disconnect হলে record update করা হয়।
3. Multiple browser tabs বা devices থাকলে connection count বিবেচনা করা হয়।
4. Unexpected disconnection-এর জন্য expiry বা heartbeat ব্যবস্থা রাখা হয়।

Important

শুধু `connect()` এবং `disconnect()` দিয়ে online status নির্ভুল রাখা যায় না।
কারণ network failure হলে disconnect event সব সময় প্রত্যাশিতভাবে নাও আসতে পারে।
Production application-এ Redis TTL, heartbeat এবং multiple connection tracking ব্যবহার করা যেতে পারে।

Part 16: Django Channels Security

50. WebSocket Security Best Practices

1. Production-এ `wss://` ব্যবহার করো।
2. Authentication যাচাই করো।
3. Room membership ও permissions যাচাই করো।
4. Origin validation implement করো।
5. Message size limit রাখো।
6. Rate limiting implement করো।
7. Client input validate করো।
8. Sensitive data log করা এড়িয়ে চলো।
9. User-supplied content HTML হিসেবে সরাসরি render করো না।
10. Redis-কে public internet-এ উন্মুক্ত রেখো না।
11. Unauthorized room access প্রতিরোধ করো।
12. Connection cleanup নিশ্চিত করো।

Origin Validation

File: `config/asgi.py`

```
import os

from django.core.asgi import get_asgi_application

from channels.routing import (
    ProtocolTypeRouter,
    URLRouter
)

from channels.auth import AuthMiddlewareStack
```

```
from channels.security.websocket import (
    AllowedHostsOriginValidator
)


os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings"
)


django_asgi_app = get_asgi_application()


from chat.routing import websocket_urlpatterns


application = ProtocolTypeRouter({

    "http": django_asgi_app,

    "websocket": AllowedHostsOriginValidator(

        AuthMiddlewareStack(

            URLRouter(websocket_urlpatterns)

        )

    ),

})
```

Production-এ `ALLOWED_HOSTS`-এ সঠিক domain configure করো।

Important: Origin validation authentication বা authorization-এর বিকল্প নয়।

Input Validation

```
message = data.get("message")


if not isinstance(message, str):

    return


if len(message) > 1000:

    return
```

Part 17: Testing Django Channels

51. WebSocket Testing

Installation:

```
pip install channels
```

File: `chat/tests.py`

```
from channels.testing import WebsocketCommunicator

from django.test import TransactionTestCase

from config.asgi import application


class EchoConsumerTests(TransactionTestCase):

    async def test_echo_connection(self):

        communicator = WebsocketCommunicator(

            application,

            "/ws/echo/"

        )


        connected, _ = await communicator.connect()

        self.assertTrue(connected)


        response = await communicator.receive_from()

        self.assertEqual(
            response,
            "WebSocket Connected!"
        )


        await communicator.send_to("Hello")


        response = await communicator.receive_from()

        self.assertEqual(
            response,
            "You sent: Hello"
        )


        await communicator.disconnect()
```

Run Tests

```
python manage.py test
```

Note: Test URL অবশ্যই তোমার actual WebSocket routing-এর সঙ্গে মিলতে হবে।

Part 18: Common WebSocket and Django Channels Errors

52. Common Errors

1. Connection Refused

Possible Causes:

- Server চালু নেই।
- ভুল port ব্যবহার করা হয়েছে।
- Endpoint ভুল।

Solution:

- Server status check করো।
- URL এবং port check করো।

2. WebSocket Closed Before Connection Established

Possible Causes:

- Server handshake reject করেছে।
- Server crash করেছে।
- Authentication failure হয়েছে।
- Routing ভুল।

Solution:

- Server logs check করো।
- Authentication এবং routing check করো।

3. 403 Forbidden

Possible Causes:

- Origin validation failure।
- Authentication failure।
- Permission failure।

Solution:

- Allowed hosts check করো।
- User authentication এবং authorization check করো।

4. 404 Not Found

Possible Cause:

WebSocket routing path ভুল।

Solution:

- `websocket_urlpatterns` check করো।
- `URLRouter` configuration check করো।

5. 500 Internal Server Error

Possible Cause:

Server-side exception।

Solution:

- Application logs check করো।
- Consumer code debug করো।

6. WebSocketDisconnect

Possible Causes:

- Client connection বন্ধ করেছে।
- Network connection lost হয়েছে।

Solution:

Disconnect event handle করো।

7. Cannot Connect to Redis

Possible Causes:

- Redis server বন্ধ।
- Host বা port ভুল।
- Redis configuration ভুল।

Solution:

Redis server status এবং configuration check করো।

8. SynchronousOnlyOperation

Possible Cause:

Async context-এ synchronous ORM ব্যবহার করা হয়েছে।

Solution:

- Async ORM methods ব্যবহার করো।
- অথবা `database_sync_to_async` ব্যবহার করো।

9. No Route Found

Possible Causes:

- WebSocket URL ভুল।
- `websocket_urlpatterns` configure করা হয়নি।
- `URLRouter` setup ভুল।

10. group_send() Not Working

Possible Causes:

- Redis/channel layer configuration ভুল।
- Consumer group-এ যোগ হয়নি।
- Event type এবং handler name মেলেনি।

Part 19: Production Deployment

53. Production Deployment Checklist

Production deployment-এর আগে:

1. `DEBUG = False` রাখো।
2. `ALLOWED_HOSTS` configure করো।
3. Environment variables-এ secrets রাখো।
4. Redis channel layer configure করো।
5. ASGI-capable server ব্যবহার করো।
6. HTTPS/WSS configure করো।
7. Nginx reverse proxy configure করো।
8. WebSocket authentication implement করো।
9. Authorization এবং room permission implement করো।
10. Logging ও monitoring setup করো।
11. Database connection এবং performance test করো।
12. Redis-এর security ও availability নিশ্চিত করো।

13. Rate limiting এবং message size limits configure করো।
14. Connection cleanup এবং reconnection handling test করো।

Development Server

```
python manage.py runserver
```

এটি development-এর জন্য।

Daphne

```
daphne -p 8000 config.asgi:application
```

Production Note

Production deployment-এ উপযুক্ত ASGI server, reverse proxy, TLS এবং shared channel layer configure করতে হবে।

Part 20: Real-World WebSocket Use Cases

54. WebSocket Use Cases

1. Real-time Chat Application.
2. Live Notification System.
3. Live Stock Market Dashboard.
4. Online Multiplayer Games.
5. Real-time Location Tracking.
6. Live Delivery Tracking.
7. Collaborative Document Editing.
8. Live Sports Score Updates.
9. Online Customer Support.
10. Real-time Monitoring Dashboard.

Part 21: Beginner to Advanced Practice Projects

55. Beginner Projects

1. Echo WebSocket Server

Features:

- Client message পাঠাবে।
- Server একই message ফিরিয়ে দেবে।

2. Simple Chat Application

Features:

- Send message.
- Receive message.
- Real-time communication.

3. JSON Message Server

Features:

- JSON request.
- JSON response.
- Input validation.

4. Basic Notification System

Features:

- Real-time notification.
- Notification message delivery.

56. Intermediate Projects

1. Group Chat Application

Features:

- Multiple rooms.
- User join/leave.
- Group messaging.
- Message history.

2. Live Notification System

Features:

- User-specific notifications.
- Unread notification count.
- Notification history.

3. Online User Status

Features:

- Online/offline status.
- Multiple devices.
- Presence tracking.

4. Private Messaging

Features:

- One-to-one chat.
- User authentication.
- Private messages.

57. Advanced Projects

1. FastAPI + Redis Chat Application

Features:

- FastAPI.
- WebSocket.
- Redis.
- PostgreSQL.
- Authentication.

2. Django Channels Chat Application

Features:

- Django Channels.
- Redis.
- PostgreSQL.
- JWT or session authentication.
- Multiple chat rooms.
- Private messaging.
- Message history.
- Read receipts.

- Online/offline status.

3. Real-Time Notification Backend

Features:

- User-specific events.
- Redis channel layer.
- Background tasks.
- Database persistence.

58. Expert Projects

- Distributed WebSocket Architecture.
- Horizontal Scaling.
- Load Balancing.
- Connection Recovery.
- Message Queue Integration.
- Production Monitoring.
- High-Concurrency WebSocket Server.
- Real-Time Collaboration Platform.

Part 22: Interview Questions and Answers

59. Beginner Level

Q1. What is WebSocket?

WebSocket is a communication protocol that provides full-duplex communication between a client and server over a persistent connection.

বাংলায়: Client এবং Server-এর মধ্যে real-time bidirectional communication তৈরি করে।

Q2. What is the difference between HTTP and WebSocket?

HTTP follows a request-response model, while WebSocket provides bidirectional communication over a persistent connection.

Q3. What is ws://?

It is the WebSocket URI scheme for an unencrypted connection.

Q4. What is wss://?

It is the secure WebSocket URI scheme that uses TLS encryption.

Q5. What is a WebSocket handshake?

It is the initial process used to establish a WebSocket connection.

Q6. What does HTTP 101 mean?

101 Switching Protocols indicates that the server accepted the protocol upgrade request.

Q7. What is full-duplex communication?

Both client and server can send and receive data independently at the same time.

60. Intermediate Level

Q8. What is Broadcasting?

Broadcasting means sending one message to multiple connected clients.

Q9. What is a WebSocket Group?

A group is a logical collection of WebSocket connections that can receive shared events.

Q10. What is Redis Channel Layer?

It is a Channels backend that uses Redis to distribute messages and events across application instances.

Q11. What is Django Channels?

Django Channels extends Django to support asynchronous protocols such as WebSocket.

Q12. What is a Consumer?

A Consumer is a Channels component that handles connection events and incoming messages.

Q13. What is the difference between SyncConsumer and AsyncConsumer?

SyncConsumer uses synchronous methods, while AsyncConsumer uses asynchronous methods and supports async I/O.

Q14. What is Ping and Pong?

Ping and Pong are WebSocket control frames used to check connection health.

Q15. What is ASGI?

ASGI stands for Asynchronous Server Gateway Interface. It provides a standard interface for asynchronous Python applications.

61. Advanced Level

Q16. How do you scale WebSocket applications?

Use multiple application servers, a load balancer, a shared messaging layer such as Redis and proper connection management.

Q17. How do you secure WebSocket?

Use WSS, authentication, authorization, origin validation, input validation, rate limiting and message size limits.

Q18. What is Exponential Backoff?

It is a reconnection strategy where the waiting time increases after each failed attempt.

Q19. What is the difference between SSE and WebSocket?

SSE provides one-way server-to-client event streaming, while WebSocket provides bidirectional communication.

Q20. Does WebSocket use TCP?

The traditional WebSocket protocol uses TCP as its transport.

Q21. Can WebSocket send binary data?

Yes. WebSocket supports both text and binary messages.

Q22. Is WebSocket stateless?

No. A WebSocket connection is stateful because it remains open and maintains connection state.

Q23. Does WebSocket replace REST API?

No. REST is useful for request-response operations, while WebSocket is useful for continuous real-time communication. They can work together.

Part 23: Important Terminology

62. WebSocket Terminology

Term	Meaning
WebSocket	Bidirectional communication protocol

Full-duplex	Simultaneous two-way communication
Handshake	Initial connection negotiation
Upgrade	Switch from HTTP to WebSocket
Persistent Connection	Connection remains open
Frame	Protocol-level data unit
Message	Data exchanged
Ping	Connection health check
Pong	Response to Ping
Broadcasting	Send to multiple clients
Consumer	Django Channels event handler
Channel Layer	Communication between consumers
Redis Pub/Sub	Publish-subscribe messaging
ASGI	Asynchronous Server Gateway Interface
WSS	Secure WebSocket
Reconnection	Re-establish a closed connection
Heartbeat	Periodic connection health check
Scope	Connection-related information
Group	Collection of channels
Daphne	ASGI server
ProtocolTypeRouter	Routes protocol types
URLRouter	Routes URL patterns
AuthMiddlewareStack	Provides session authentication

Part 24: Complete Learning Roadmap

63. WebSocket Learning Roadmap

Level	Topics
01	HTTP Basics
02	Client-Server Architecture
03	WebSocket Fundamentals
04	WebSocket Handshake
05	Python Async Programming
06	WebSocket Server
07	WebSocket Client
08	JSON Communication
09	FastAPI WebSocket
10	Connection Manager
11	Broadcasting
12	Authentication
13	Authorization
14	Ping/Pong
15	Reconnection
16	Redis Pub/Sub
17	Django Channels
18	Consumers
19	WebSocket Routing
20	Channel Layer
21	Groups
22	Database Integration
23	Security
24	Testing
25	Production Deployment

64. Django Channels Learning Roadmap

Level 1: Fundamentals

- Django Fundamentals.

- HTTP Basics.
- Client-Server Architecture.
- WebSocket Basics.
- WSGI vs ASGI.

Level 2: Django Channels Setup

- Installation.
- Project Structure.
- settings.py.
- asgi.py.
- Consumers.
- WebSocket Routing.

Level 3: WebSocket Communication

- Connect.
- Disconnect.
- Receive.
- Send.
- JSON Communication.
- Error Handling.

Level 4: Channel Layer and Redis

- Channel Layer.
- Redis Fundamentals.
- Redis Configuration.
- Groups.
- Broadcasting.

Level 5: Advanced Features

- Authentication.
- Authorization.
- Private Rooms.
- Database Integration.
- Notifications.
- Typing Indicators.
- Online Presence.

Level 6: Production

- Testing.
- Security.
- Performance.
- Rate Limiting.
- Monitoring.
- Deployment.
- Scaling.

Part 25: 12-Week Learning Plan

65. 12-Week Roadmap

Week	Topics
Week 1	Python Async/Await and asyncio
Week 2	HTTP, WebSocket and ASGI
Week 3	Python WebSocket Server and Client
Week 4	FastAPI WebSocket
Week 5	Django Channels and Consumers
Week 6	Routing and Channel Layer
Week 7	Redis and Groups
Week 8	Authentication and Authorization
Week 9	Database Integration
Week 10	Notifications, Typing and Presence
Week 11	Security, Testing and Performance
Week 12	Production Deployment and Final Project

Part 26: Important Commands

66. Python WebSocket

Install:

```
pip install websockets
```

67. FastAPI

Install:

```
pip install fastapi uvicorn
```

Run:

```
uvicorn main:app --reload
```

68. Django Channels

Install:

```
pip install django channels
```

69. Redis Channel Layer

Install:

```
pip install channels-redis
```

70. Daphne

Install:

```
pip install daphne
```

Run:

```
daphne -p 8000 config.asgi:application
```

71. Django Commands

Create project:

```
django-admin startproject config .
```

Create app:

```
python manage.py startapp chat
```

Migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Run development server:

```
python manage.py runserver
```

Run tests:

```
python manage.py test
```

Part 27: Official Documentation

72. Recommended Documentation

1. Python Asyncio:
   https://docs.python.org/3/library/asyncio.html
2. Python WebSockets:
   https://websockets.readthedocs.io/
3. FastAPI WebSockets:
   https://fastapi.tiangolo.com/advanced/websockets/
4. Django:
   https://docs.djangoproject.com/
5. Django Channels:
   https://channels.readthedocs.io/
6. Django Channels Tutorial:
   https://channels.readthedocs.io/en/stable/tutorial/index.html
7. Django Channels Deployment:
   https://channels.readthedocs.io/en/stable/deploying.html
8. Redis:
   https://redis.io/docs/latest/

9. MDN WebSocket API:
   https://developer.mozilla.org/en-US/docs/Web/API/WebSocket
10. WebSocket Protocol RFC 6455:
    https://www.rfc-editor.org/rfc/rfc6455
11. ASGI:
    https://asgi.readthedocs.io/

Part 28: Final Summary

73. Most Important WebSocket Concepts

1. WebSocket provides bidirectional communication.
2. WebSocket connection is persistent.
3. WebSocket starts with a handshake.
4. HTTP 101 means protocol switching accepted.
5. `ws://` is unencrypted.
6. `wss://` is encrypted.
7. FastAPI supports WebSocket directly.
8. Django uses Channels for WebSocket support.
9. Redis helps distribute messages between servers.
10. Authentication and authorization are essential.
11. Production applications should use WSS.
12. WebSocket does not replace REST API.

74. Most Important Django Channels Concepts

1. ASGI is the asynchronous application interface.
2. Consumer handles WebSocket events.
3. Routing connects URLs with Consumers.
4. `connect()` handles new connections.
5. `disconnect()` handles disconnections.
6. `receive()` handles incoming messages.
7. `send()` sends messages.
8. Channel Layer handles inter-consumer communication.
9. Redis provides shared messaging.
10. Groups support broadcasting.
11. `scope` contains connection information.
12. `AuthMiddlewareStack` provides session-based authentication.
13. `database_sync_to_async` supports synchronous ORM operations in async consumers.
14. Authentication and authorization must both be implemented.
15. Testing and security are essential for production.

75. Recommended Final Project

Build a Complete Real-Time Chat Application using Django Channels + Redis + PostgreSQL.

Project Features

- User Registration and Login.
- Session or JWT Authentication.
- One-to-One Private Chat.

- Multiple Chat Rooms.
- Group Messaging.
- Real-Time Notifications.
- Typing Indicators.
- Online/Offline Status.
- Message History.
- Read Receipts.
- Message Timestamps.
- Redis Channel Layer.
- PostgreSQL Database.
- Input Validation.
- Rate Limiting.
- Automated Testing.
- Production Deployment.

Recommended Learning Sequence

```
Python Async Programming
          |
          v
WebSocket Fundamentals
          |
          v
Python WebSocket
          |
          v
FastAPI WebSocket
          |
          v
Django Channels
          |
          v
Consumers and Routing
          |
          v
Channel Layer
          |
          v
Redis and Groups
          |
          v
Authentication and Authorization
          |
          v
PostgreSQL Integration
          |
          v
Real-Time Chat Application
          |
          v
Testing and Security
```

```
            |
            v
  Production Deployment
```

Final Advice:

প্রথমে EchoConsumer তৈরি করে WebSocket connection বুঝো। এরপর Redis এবং Group Chat implement করো। তারপর Authentication, Database Persistence, Private Rooms, Automated Tests এবং Production Deployment শেখো।

প্রতিটি project-এ authentication, authorization, input validation, testing এবং error handling যোগ করার চেষ্টা করো।

এভাবে ধাপে ধাপে practice করলে Python WebSocket এবং Django Channels Beginner থেকে Advanced Level পর্যন্ত শেখা সহজ হবে।

END OF PYTHON WEBSOCKET & DJANGO CHANNELS COMPLETE NOTES